Chapter 6

Web Application Deployment and Modern Trends

1. Full-stack development
2. Testing and QA
3. DevOps, continuous integration (CI) and continuous delivery (CD)
4. Progressive web apps (PWAs), responsive design and usability

Time Duration: 5 Hours
Marks: 7

1. Full-Stack Development

Meaning:

Full-stack development means building both:

- **Frontend (Client Side)** → what users see and interact with
- **Backend (Server Side)** → logic, database, authentication, APIs

A **full-stack developer** can work with both sides.

A) Frontend Development

Role of Frontend

Frontend is responsible for:

- UI design (buttons, forms, layout)
- User experience (smooth navigation, responsiveness)
- Communicating with backend using APIs
- Displaying data dynamically

Frontend Technologies

- **HTML** → structure
- **CSS / Tailwind / Bootstrap** → styling
- **JavaScript / TypeScript** → interactivity
- **React / Next.js / Vue / Angular** → modern frontend frameworks

Frontend Output

Frontend sends requests like:

- Login request
- Fetch products list
- Upload file request

B) Backend Development

Role of Backend

Backend is responsible for:

- Handling user requests
- Authentication and authorization
- Business logic
- Database operations (CRUD)
- API development

Backend Technologies

- Node.js + Express.js
- Django / Flask (Python)
- Spring Boot (Java)
- ASP.NET Core (C#)

Backend API Types

- REST API (most common)
- GraphQL API (modern, flexible)
- WebSockets (real-time apps)

C) Database Layer

✔ Role of Database

Stores application data such as:

- user info
- posts
- orders
- payment details
- logs

Types of Databases

Relational Databases (SQL)

- MySQL
- PostgreSQL
- Oracle
- SQL Server

Used when data is structured (tables, relations).

NoSQL Databases

- MongoDB
- Firebase
- Cassandra

Used when data is flexible (documents, JSON, big data).

D) Full Stack Architecture Example

Example: **E-commerce Website**

- Frontend: React + Tailwind
- Backend: Node + Express
- Database: PostgreSQL
- Hosting: Vercel + Render
- Storage: AWS S3, cloudinary
- Authentication: JWT / OAuth /session based

E) Modern Full-Stack Trends

1. JAMstack

- JavaScript + APIs + Markup
- Uses static generation + serverless backend

Example: Next.js + Firebase

2. Serverless Backend

Instead of managing servers, you use cloud functions:

- AWS Lambda
- Firebase Functions
- Vercel Functions

Benefits:

- scalable
- cheaper
- no server management

3. Microservices Architecture

Instead of one big backend (monolith), system is divided into services:

- Auth Service
- Payment Service
- Product Service

Benefits:

- easy scaling
- independent deployment

Testing and QA (Quality Assurance)

Meaning

Testing checks if the application works correctly.
QA ensures overall product quality including performance, usability, and security.

Goal: **detect bugs early and improve reliability.**

A) Why Testing is Important?

- prevents bugs in production
- improves security and stability
- saves time and cost
- increases user trust
- ensures smooth deployment

B) Types of Testing

1. Unit Testing

Tests individual functions/components.

Example:

- checking if calculateTotal() returns correct value
- testing login validation function

Tools:

- Jest (JavaScript)
- Mocha + Chai
- PyTest (Python)

2. Integration Testing

Tests whether modules work together.

Example:

- frontend login form + backend login API working together
- database + backend API working properly

Tools:

- Jest + Supertest
- Postman tests

3. System Testing

Tests the complete application as a whole.

Example:

- full website flow: signup → login → order → payment → logout

4. End-to-End (E2E) Testing

Simulates real user behavior.

Example:

- open browser → enter username/password → click login → verify dashboard loads

Tools:

- Cypress
- Selenium
- Playwright

5. Manual Testing

Human testers test UI manually.

- checking forms
- checking design
- verifying navigation

6. Automated Testing

Test scripts run automatically.

Benefits:

- fast testing
- repeatable
- good for CI/CD pipelines

7. Performance Testing

Checks speed, scalability, load handling.

Example:

- Can the system handle 10,000 users?

Tools:

- JMeter
- Locust
- Lighthouse

8. Security Testing

Checks vulnerabilities like:

- SQL Injection
- XSS
- CSRF
- broken authentication

Tools:

- OWASP ZAP
- Burp Suite

C) QA Process Steps

1. Requirement Analysis
2. Test planning
3. Test case writing
4. Test execution
5. Bug reporting
6. Re-testing
7. Final verification

D) Bug Life Cycle

1. Bug Found
2. Bug Reported
3. Developer Fixes
4. Tester Re-tests
5. Bug Closed

3. DevOps, CI (Continuous Integration) and CD (Continuous Delivery/Deployment)

Meaning :

DevOps = Development + Operations

It is a practice that connects:

- developers (build software)
- operations team (deploy and maintain software)

Goal: **fast development + fast deployment + stable production.**

A) Why DevOps is Needed?

Traditional approach had problems:

- slow deployment
- conflicts between dev and ops teams
- bugs in production
- manual server management

DevOps solves these by:

- automation
- monitoring
- fast deployment pipelines

B) Key DevOps Practices

1. Version Control

Tracks changes in code.

Tool: Git
Platforms:

- GitHub
- GitLab
- Bitbucket

2. Build Automation

Compiling and packaging the application automatically.

Example:

- React app build: `npm run build`
- Node app bundling
- Docker image building

3. Infrastructure as Code (IaC)

Servers are managed using code instead of manual configuration.

Tools:

- Terraform
- Ansible
- CloudFormation

4. Monitoring and Logging

Tracking application health and errors.

Tools:

- Prometheus
- Grafana
- ELK Stack
- Sentry

CI (Continuous Integration)

Meaning

CI means **automatically merging and testing code** whenever developers push changes.

Example flow:

- Developer pushes code to GitHub
- Automated tests run
- If tests pass, merge allowed

Benefits of CI

- detects bugs early
- reduces integration conflicts
- improves team collaboration

Tools:

- GitHub Actions
- Jenkins
- GitLab CI
- CircleCI

CD (Continuous Delivery / Continuous Deployment)

Continuous Delivery

Code is always ready for deployment.
Deployment requires manual approval.

Example:

- build + test + package automatically
- deployment happens when admin clicks "Deploy"

Continuous Deployment

Deployment happens automatically after tests pass.

Example:

- push code → tests pass → deployed to server instantly

CI/CD Pipeline Example

Example: React + Node Application

1. Developer pushes code to GitHub
2. CI runs unit tests
3. Build frontend + backend
4. Create Docker image
5. Deploy to cloud server (AWS, DigitalOcean)
6. Monitoring begins

Deployment Platforms Used Today

Frontend Hosting

- Vercel (best for Next.js)
- Netlify
- GitHub Pages

Backend Hosting

- Render
- Railway
- AWS EC2
- DigitalOcean
- Heroku (less popular now)

Database Hosting

- MongoDB Atlas
- Supabase (PostgreSQL)
- PlanetScale (MySQL)
- NeonDB (PostgreSQL)

Docker in Deployment

Meaning

Docker packages application with dependencies into a container.

Benefits:

- works same everywhere (local, server, cloud)
- easy deployment
- avoids "works on my machine" problem

4. Progressive Web Apps (PWAs), Responsive Design and Usability

A) Progressive Web Apps (PWAs)

Meaning

A **Progressive Web App** is a website that behaves like a mobile app.

It can:

- work offline
- be installed on phone
- send notifications
- load faster

Key Features of PWA

1. Installable

Users can install it like an app from browser.

2. Offline Support

Works even without internet using caching.

3. Fast Performance

Uses caching and service workers.

4. Push Notifications

Can notify users about updates.

5. Secure (HTTPS)

PWAs require HTTPS for security.

Technologies Used in PWA

1. **Service Worker**

   A script running in background that handles:

   - caching
   - offline support
   - push notifications

2. **Web App Manifest**

   A JSON file defining:

   - app name
   - icon
   - theme color
   - Start URL

PWA Example

Example: News website PWA

- loads news even offline
- user installs it
- notifications for breaking news

Advantages of PWA

- no need to download from Play Store
- works on all devices
- cheaper than native app development
- lightweight and fast

Limitations of PWA

- limited access to device hardware compared to native apps
- some iOS restrictions
- not all features supported equally in all browsers

B) Responsive Design

Meaning

Responsive design means the website adapts to different screen sizes:

- mobile
- tablet
- laptop
- desktop

Goal: **same website works everywhere.**

Why Responsive Design is Important?

- majority users use mobile devices
- improves user experience
- improves SEO ranking (Google prefers mobile-friendly sites)
- reduces need for separate mobile website

Key Concepts in Responsive Design

1. Flexible Layouts

Using percentage widths instead of fixed px.

Example: width: 100% instead of width: 500px

2. Media Queries

CSS rules that apply based on screen size.

Example:

```
@media (max-width: 768px) {
  .menu {
    display: none;
  }
}
```

3. Responsive Images

Images scale automatically.

Example:

```
img {
  max-width: 100%;
  height: auto;
}
```

4. Mobile-First Design

Design for mobile first, then expand for desktop.

This is the modern recommended approach.

Responsive Tools

- Tailwind CSS responsive classes:
  - sm:
  - md:
  - lg:
  - xl:

Example:

```
<div class="text-sm md:text-lg lg:text-xl">

  Hello

</div>
```

C) Usability (User-Friendly Design)

Meaning

Usability refers to how easily users can use your application.

A usable website is:

- simple
- clear
- fast
- accessible

Factors of Good Usability

1. Easy Navigation

- menus should be clear
- important pages should be reachable quickly

Bad example:

- user needs 6 clicks to reach contact page

2. Consistent UI

- same button styles everywhere
- same fonts, colors
- predictable layout

3. Readability

- good font size
- proper spacing
- proper contrast between text and background

4. Fast Loading

Users leave slow websites quickly.

Modern expectation:

- site should load in under 3 seconds

5. Accessibility

Website should be usable by disabled users.

Examples:

- alt text for images
- keyboard navigation
- proper headings
- readable colors

---

6. Error Handling and Feedback

Users should know what is happening.

Example:

- show error if password is incorrect
- show loading spinner during API calls
- show success message after form submission

7. Minimalistic Design

Too many elements confuse users.

Rule: keep UI clean and focused.

Modern Trends in Web Deployment and Development

1. Cloud Deployment

Instead of local servers, apps are hosted on cloud platforms:

- AWS
- Google Cloud
- Microsoft Azure

Benefits:

- scalability
- reliability
- global access

2. Containerization (Docker + Kubernetes)

Docker runs applications in containers.
Kubernetes manages multiple containers.

Used in large applications like:

- Netflix
- Amazon

3. API-First Development

Backend is designed as API first, so it supports:

- web app
- mobile app
- desktop app

4. Microservices and Distributed Systems

Large systems are broken into smaller services.

5. AI and Automation in Web Apps

Modern web apps include:

- AI chatbots
- recommendation systems
- automated testing using AI tools

6. Headless CMS

Backend content is managed separately and served via API. Examples:

- Strapi
- Contentful
- Sanity

Used for blogs, news websites.

Presentation Topics for Chapter-6 (Deployment)

S.N	Topics	Presenter	Remarks
1	Full-Stack Web Application Architecture (Frontend + Backend + Database flow)	14	Done
2	REST API vs GraphQL in Modern Web Apps	32	Done
3	Database Hosting and Cloud Databases (MongoDB Atlas, Supabase, etc.)	1	Done
4	Deployment of Frontend Applications (Vercel, Netlify, GitHub Pages)	33	Done
5	Backend Deployment Strategies (Render, Railway, AWS, DigitalOcean)	42	Done
6	Docker and Containerization for Web Deployment	26	Done
7	CI/CD Pipeline Concepts and Workflow	15	Done
8	DevOps Tools and Practices (Git, GitHub Actions, Monitoring)	36	Done
9	Testing in Web Development: Unit, Integration, End-to-End Testing	3	Done
10	Automated Testing Tools (Jest, Cypress, **Selenium**)	40	Done
11	Progressive Web Apps (PWA): Features and Working	30	Done
12	Service Workers and Caching Mechanisms in PWA		
13	Responsive Web Design Principles (Mobile-first, Media Queries, Grid/Flex)	41	Done
14	Web Usability and UI/UX Best Practices	45	
15	Web Performance Optimization (**Lazy loading**, **CDN**, compression, caching)	29	